
Soft Actor-Critic (SAC) Algorithm Notes

千喜 Qx

2026/06/09

Contents

1	Background and Motivation	1
1.1	Limitations of Standard Reinforcement Learning	1
1.2	Maximum Entropy Reinforcement Learning Framework	1
1.3	Why Entropy Regularization?	1
2	The Maximum Entropy Objective.	2
2.1	Standard RL Objective	2
2.2	Entropy-Augmented Objective	2
2.3	The Temperature Parameter α	2
2.4	Shannon Entropy Definition	3
3	Soft Value Functions	3
3.1	Soft Q-Function: Definition and Intuition	3
3.2	Soft V-Function: Definition and Intuition	3
3.3	Relationship Between Soft Q and Soft V	3
3.4	The Soft Bellman Equation	3
4	Soft Policy Iteration.	4
4.1	Overview: Two-Step Alternating Process	4
4.2	Soft Policy Evaluation: The Contraction Operator	4
4.3	Soft Policy Improvement: Constrained Optimization	4
4.4	The Partition Function $Z(s)$ and Normalization	4
4.5	KL Divergence Projection (Information Projection)	5
4.6	Mathematical Proof of Monotonic Improvement	5
5	Function Approximation	5
5.1	Neural Network Architecture	6

5.2	Critic Network Loss (Q-Network)	6
5.3	Actor Network Loss (π -Network)	6
5.4	The Reparameterization Trick	6
6	The Tanh Correction Term	7
6.1	Why We Need Tanh Squashing	7
6.2	Change-of-Variables Formula	7
6.3	Jacobian Determinant for Diagonal Gaussian	7
6.4	Deriving the Final Log-Likelihood	7
6.5	Stable Numerical Implementation	8
7	Automatic Entropy Tuning	8
7.1	Constrained Formulation	8
7.2	Lagrange Multiplier Method	8
7.3	The α Update Loss Function	8
7.4	Dynamics of α Adaptation	8
7.5	Choosing Target Entropy $\bar{H}$	9
8	The Partition Function $Z(s)$ in Practice	9
8.1	Physics Origin: Boltzmann Distribution	9
8.2	Why $Z(s)$ Cannot Be Computed Directly	10
8.3	The Gradient Vanishing Trick	10
9	Actor-Critic Architecture Analysis	10
9.1	Why Both Q-Network and V-Network? (Original Design)	10
9.2	Why One Network Is Enough (SAC-v2)	10
9.3	The Circular Dependency Problem	11
9.4	Target Network and Polyak Averaging	11
10	Twin Q-Networks and Overestimation Bias	11
10.1	The Maximization Bias Problem	11
10.2	Mathematical Analysis of Error Propagation	11
10.3	The Minimum Q-Value Trick (Clipped Double-Q)	12
11	Off-Policy vs On-Policy Analysis	12
11.1	Why SAC Can Be Off-Policy	12
11.2	Why PPO Is Inherently On-Policy	12
11.3	Key Architectural Differences	13
12	Implementation Details and Engineering Tricks	13
12.1	Why Predict <code>log_std</code> Instead of <code>std</code>	13
12.2	Clamping <code>log_std</code> : <code>[-20, 2]</code> Rationale	13
12.3	The <code>.sum(dim=-1, keepdim=True)</code> Operation	13
12.4	Exploration vs Evaluation Mode	14
13	Complete PyTorch Implementation	14
13.1	Replay Buffer	14
13.2	Network Definitions	15
13.3	SAC Agent	16
14	Practical Projects	18

14.1	Quadcopter / Drone Control	18
14.2	Autonomous Driving & Lane Keeping	18
14.3	Bipedal Locomotion	18
14.4	Financial Portfolio Optimization	18
15	Training Diagnostics	19
15.1	Understanding Reward Vibration	19
15.2	The Role of Randomized Initial States	19
15.3	How to Verify True Convergence	19
16	Study Quiz	19
16.1	Question 1: Core Optimization Objective	19
16.2	Question 2: The Temperature Parameter (α)	20
16.3	Question 3: Twin Q-Functions	20
16.4	Question 4: Continuous Action Spaces	20
16.5	Question 5: Gradient Variance	21
16.6	Question 6: Soft Q-Value Target	21
16.7	Question 7: Automatic Temperature Adjustment	21
16.8	Question 8: Algorithm Type	21
16.9	Question 9: Benefits of Maximum Entropy	22
16.10	Question 10: Target Network Updates	22
A	Appendix A: Mathematical Foundations	22
A.1	Kullback-Leibler Divergence Properties	22
A.2	Change-of-Variables Formula (General Form)	23
A.3	Softplus Function Stable Implementation	23
B	Appendix B: Symbol Table	23
C	Appendix C: Core Formula Summary	24

1 Background and Motivation

1.1 Limitations of Standard Reinforcement Learning

Standard reinforcement learning algorithms aim to maximize the expected cumulative reward:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^T r(s_t, a_t) \right]$$

Limitations:

- **Poor Exploration:** Standard RL agents tend to converge to local optima quickly, especially in sparse reward environments.
- **Sensitivity to Initial Conditions:** The learned policy can be heavily dependent on the initial state distribution and early experiences.
- **Lack of Robustness:** A policy that is optimal for one task distribution may completely fail when the environment changes slightly.

1.2 Maximum Entropy Reinforcement Learning Framework

Soft Actor-Critic (SAC), derived by Tuomas Haarnoja et al. in 2018, is an **off-policy actor-critic algorithm** based on the **maximum entropy reinforcement learning framework**.

Unlike standard RL, which only maximizes expected cumulative rewards, SAC encourages the agent to act as randomly as possible while still succeeding at the task:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^T r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t)) \right]$$

This leads to much better exploration and robustness.

1.3 Why Entropy Regularization?

The entropy $\mathcal{H}(\pi)$ of a policy measures the “randomness” or “uncertainty” in the action selection:

- **High entropy** $\implies$ The policy is more exploratory, spreading probability mass across many actions.
- **Low entropy** $\implies$ The policy is more deterministic, focusing on specific actions.

Benefits:

1. **Explicit Exploration Incentive:** The agent is explicitly rewarded for maintaining uncertainty, preventing premature convergence.
2. **Multi-Modal Discovery:** In tasks with multiple equally good strategies, maximum entropy RL can learn to use all of them.
3. **Improved Stability:** The entropy term smooths the optimization landscape.

2 The Maximum Entropy Objective

2.1 Standard RL Objective

In standard RL, the goal is to maximize the expected sum of rewards:

$$J(\pi) = \mathbb{E}_{(s_t, a_t) \sim \rho_\pi} \left[\sum_{t=0}^T r(s_t, a_t) \right]$$

where ρ_π is the state-action trajectory distribution under policy π .

2.2 Entropy-Augmented Objective

In maximum entropy RL, we add an entropy regularization term. The **generalized objective** $J(\pi)$ is defined as:

$$J(\pi) = \mathbb{E}_{(s_t, a_t) \sim \rho_\pi} \left[\sum_{t=0}^T r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t)) \right] \quad (1)$$

Symbol Definitions:

- ρ_π : State-action trajectory distribution under policy π
- α : Temperature parameter controlling the trade-off between reward and entropy
- $\mathcal{H}(\pi(\cdot | s_t))$: Shannon entropy of the policy at state s_t

2.3 The Temperature Parameter α

The temperature parameter α controls the **trade-off** between:

- **Maximizing reward** (exploitation): Focusing on actions known to yield high returns.
- **Maximizing entropy** (exploration): Acting as randomly as possible.

Effect of α values:

Table 1: Effect of α values	
α Value	Behavior
$\alpha = 0$	Reduces to standard RL; no entropy bonus
$\alpha \rightarrow \infty$	Policy becomes purely random (uniform distribution)
α small but positive	Balances exploitation and exploration

2.4 Shannon Entropy Definition

The Shannon entropy of the policy at state s_t is calculated as:

$$\mathcal{H}(\pi(\cdot|s_t)) = \mathbb{E}_{a_t \sim \pi(\cdot|s_t)}[-\log \pi(a_t|s_t)] \quad (2)$$

Intuition: The entropy is the expected negative log-probability of the action taken. If the policy is deterministic (always selects one action with probability 1), then $-\log(1) = 0$ and entropy is zero. If the policy is uniform (all actions equally likely), entropy is maximized.

3 Soft Value Functions

3.1 Soft Q-Function: Definition and Intuition

The soft Q-value represents the expected return **plus** the expected future entropy of following policy π after taking action a_t in state s_t :

$$Q(s_t, a_t) = r(s_t, a_t) + \mathbb{E}_{\tau \sim \pi} \left[\sum_{k=1}^{T-t} \gamma^k (r(s_{t+k}, a_{t+k}) + \alpha \mathcal{H}(\pi(\cdot|s_{t+k}))) \right] \quad (3)$$

Key insight: Unlike standard Q-values which only count future rewards, soft Q-values also account for future entropy bonuses.

3.2 Soft V-Function: Definition and Intuition

The soft V-value represents the expected total value (including entropy) of starting in state s_t :

$$V(s_t) = \mathbb{E}_{a_t \sim \pi(\cdot|s_t)}[Q(s_t, a_t) - \alpha \log \pi(a_t|s_t)] \quad (4)$$

3.3 Relationship Between Soft Q and Soft V

By substituting the definition of $V(s_{t+1})$ into the Q-function:

$$V(s_t) = \mathbb{E}_{a_t \sim \pi}[Q(s_t, a_t) - \alpha \log \pi(a_t|s_t)]$$

This shows that the soft V-function is the **expectation** of the soft Q-function under the current policy, minus the entropy term.

3.4 The Soft Bellman Equation

By substituting $V(s_{t+1})$ back into the Q-function, we derive the **Soft Bellman Equation**:

$$Q(s_t, a_t) = r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1} \sim p}[V(s_{t+1})] \quad (5)$$

Comparison with Standard Bellman Equation:

Table 2: Standard vs Soft Bellman Equation

Aspect	Standard RL	Soft RL
Q-function update	$Q(s, a) = r + \gamma \mathbb{E}[Q(s', a')]$	$Q(s, a) = r + \gamma \mathbb{E}[V(s')]$
Key difference	Q depends on future Q-values	Q depends on V (expectation over actions)

The key difference is that the soft V-function already includes the entropy term, so we don't need to sample an action in the backup.

4 Soft Policy Iteration

4.1 Overview: Two-Step Alternating Process

SAC guarantees convergence to the optimal maximum entropy policy by **alternating** between two steps:

1. **Soft Policy Evaluation:** Compute the soft Q-value for an arbitrary policy π .
2. **Soft Policy Improvement:** Update the policy toward the optimal distribution.

4.2 Soft Policy Evaluation: The Contraction Operator

We define a **modified Bellman backup operator** $\mathcal{T}^\pi$:

$$\boxed{\mathcal{T}^\pi Q(s, a) \triangleq r(s, a) + \gamma \mathbb{E}_{s' \sim p}[V(s')]} \quad (6)$$

where $V(s') = \mathbb{E}_{a' \sim \pi}[Q(s', a') - \alpha \log \pi(a'|s')]$.

Contraction Mapping Theorem: Repeatedly applying $\mathcal{T}^\pi$ causes Q to **converge monotonically** to the true soft Q-value Q^π for that policy.

4.3 Soft Policy Improvement: Constrained Optimization

In the improvement step, we want to find a new policy π_{new} that **maximizes** the expected soft value:

$$\pi_{\text{new}} = \arg \max_{\pi' \in \Pi} \mathbb{E}_{a \sim \pi'} [Q^{\pi_{\text{old}}}(s, a) - \alpha \log \pi'(a|s)] \quad (7)$$

4.4 The Partition Function $Z(s)$ and Normalization

The Boltzmann (Gibbs) distribution assigns probabilities exponentially proportional to Q-values:

$$\pi^*(a|s) \propto \exp\left(\frac{1}{\alpha} Q(s, a)\right)$$

To make this a valid probability distribution, we need to normalize:

$$Z(s) = \int_{\mathcal{A}} \exp\left(\frac{1}{\alpha} Q(s, a')\right) da' \quad (8)$$

Why We Need $Z(s)$: An exponential function outputs values from 0 to ∞ . For a valid probability density, the total integral must equal 1.

4.5 KL Divergence Projection (Information Projection)

To solve the constrained optimization, we **project** the exponential onto Π using the **Kullback-Leibler (KL) divergence**:

$$\pi_{\text{new}} = \arg \min_{\pi' \in \Pi} D_{\text{KL}} \left(\pi'(\cdot|s) \parallel \frac{Z(s) \exp\left(\frac{1}{\alpha} Q^{\pi_{\text{old}}}(s, \cdot)\right)}{Z(s)} \right) \quad (9)$$

Decoding the Original SAC Paper

“To account for the constraint that $\pi \in \Pi$, we project the improved policy into the desired set of policies. While in principle we could choose any projection, it will turn out to be convenient to use the information projection defined in terms of the Kullback-Leibler divergence.”

Decoded:

1. **“The constraint $\pi \in \Pi$ ”:** Our neural network can only output simple shapes (e.g., Gaussian distributions).
2. **“Project”:** Find the closest distribution within Π to the theoretical optimal π^* .
3. **“Information projection”:** Use KL divergence in a specific direction ($D_{\text{KL}}(\pi_{\phi} \parallel \pi^*)$).

4.6 Mathematical Proof of Monotonic Improvement

Expanding the KL Divergence:

$$D_{\text{KL}}(\pi' \parallel \pi^*) = \mathbb{E}_{a \sim \pi'} \left[\log \pi'(a|s) - \frac{1}{\alpha} Q^{\pi_{\text{old}}}(s, a) + \log Z(s) \right]$$

Since $\log Z(s)$ depends only on the state (not on π'), it acts as a **constant** during optimization. **Minimizing this KL divergence is mathematically identical to maximizing:**

$$\mathbb{E}_{a \sim \pi'} [Q^{\pi_{\text{old}}}(s, a) - \alpha \log \pi'(a|s)]$$

Key Conclusion 4.1— *Updating the policy via KL divergence minimization **strictly improves** its soft value function.*

5 Function Approximation

5.1 Neural Network Architecture

In continuous, high-dimensional spaces, we use neural networks:

Table 3: Network Architecture

Network	Symbol	Function
Critic	$Q_\theta(s, a)$	Approximates the soft Q-value
Actor	$\pi_\phi(a s)$	Outputs the policy distribution

5.2 Critic Network Loss (Q-Network)

The parameters θ are trained by minimizing the **mean squared Bellman error**:

$$J_Q(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\frac{1}{2} (Q_\theta(s, a) - y)^2 \right] \quad (10)$$

where the **target value function** $\hat{V}(s')$ is:

$$\hat{V}(s') = \mathbb{E}_{a' \sim \pi_\phi(\cdot|s')} [Q_{\bar{\theta}}(s', a') - \alpha \log \pi_\phi(a'|s')]$$

Target Value Calculation:

$$y = r + \gamma \hat{V}(s')$$

5.3 Actor Network Loss (π -Network)

The actor parameters ϕ are trained directly to minimize the KL divergence objective:

$$J_\pi(\phi) = \mathbb{E}_{s \sim \mathcal{D}, a \sim \pi_\phi} [\alpha \log \pi_\phi(a|s) - Q_\theta(s, a)] \quad (11)$$

5.4 The Reparameterization Trick

Problem: The stochastic sampling $a \sim \pi_\phi(\cdot|s)$ creates a **high-variance** gradient estimator.

Solution: Apply the **reparameterization trick**. We define the action as a **deterministic function** of the state and an independent noise vector ϵ :

$$a = f_\phi(\epsilon; s) = \tanh(\mu_\phi(s) + \sigma_\phi(s) \odot \epsilon), \quad \epsilon \sim \mathcal{N}(0, I) \quad (12)$$

Why this works: The noise ϵ is independent of ϕ . Gradients can now flow directly through μ_ϕ and σ_ϕ because the stochasticity is isolated in ϵ .

Reparameterized Actor Loss:

$$J_\pi(\phi) = \mathbb{E}_{s \sim \mathcal{D}, \epsilon \sim \mathcal{N}} [\alpha \log \pi_\phi(f_\phi(\epsilon; s)|s) - Q_\theta(s, f_\phi(\epsilon; s))]$$

6 The Tanh Correction Term

6.1 Why We Need Tanh Squashing

Neural networks naturally output unbounded values. However, physical environments have **hard constraints**. To enforce this, SAC passes the raw Gaussian policy output through a **tanh activation function** to bound actions to $[-1, 1]^D$.

6.2 Change-of-Variables Formula

Problem: Because the action distribution is squashed by tanh, we cannot simply calculate the log-likelihood using the standard Gaussian formula.

Setup:

- $u \in \mathbb{R}^D$: Raw unbounded action sampled from Gaussian distribution $\mu(u|s)$
- $a = \tanh(u)$: Squashed, final action bounded within $(-1, 1)^D$

The probability density of the squashed action $\pi(a|s)$ is related to the base density $\mu(u|s)$ by:

$$\pi(a|s) = \mu(u|s) \cdot \left| \det \left(\frac{du}{da} \right) \right|^{-1}$$

6.3 Jacobian Determinant for Diagonal Gaussian

Because tanh is applied **element-wise** to each action dimension independently, the Jacobian is a **diagonal matrix**:

$$\frac{du_i}{da_i} = \frac{1}{\text{sech}^2(u_i)} = 1 - \tanh^2(u_i)$$

The determinant of a diagonal matrix is simply the product of its diagonal elements:

$$\det \left(\frac{du}{da} \right) = \prod_{i=1}^D (1 - \tanh^2(u_i))$$

6.4 Deriving the Final Log-Likelihood

Plugging this back into the change-of-variables equation:

$$\pi(a|s) = \mu(u|s) \cdot \prod_{i=1}^D \frac{1}{1 - \tanh^2(u_i)}$$

Taking the natural logarithm:

$$\log \pi(a|s) = \log \mu(u|s) - \sum_{i=1}^D \log (1 - \tanh^2(u_i)) \quad (13)$$

This is the exact correction term used in SAC.

Confusion Point: Why the Correction is Necessary

Some practitioners forget to apply the tanh correction, which leads to:

1. **Incorrect Policy Gradient:** The gradients don't properly account for the bounded action space.
2. **Policy Collapse:** The agent may output actions outside the valid range when the environment clips them.

6.5 Stable Numerical Implementation

Problem: When $u = 10$, $\tanh(10) \approx 1$, making $1 - \tanh^2(u) \approx 0$, and $\log(0) \rightarrow -\infty$.

Solution: Use the stable form:

$$\log(1 - \tanh^2(u)) = 2 \log 2 - u - \text{softplus}(-2u) \quad (14)$$

where $\text{softplus}(x) = \log(1 + e^x)$.

7 Automatic Entropy Tuning

7.1 Constrained Formulation

Instead of fixing α as a constant, we reframe maximum entropy RL as a **constrained optimization problem**:

$$\max_{\pi_{0:T}} \mathbb{E} \left[\sum_{t=0}^T r(s_t, a_t) \right] \quad \text{s.t.} \quad \mathcal{H}(\pi_t(\cdot|s_t)) \geq \bar{\mathcal{H}} \quad \forall t$$

7.2 Lagrange Multiplier Method

Using the method of **Lagrange Multipliers**:

$$\mathcal{L}(\pi_{0:T}, \alpha_{0:T}) = \mathbb{E} \left[\sum_{t=0}^T r(s_t, a_t) + \alpha_t (\mathcal{H}(\pi_t(\cdot|s_t)) - \bar{\mathcal{H}}) \right]$$

7.3 The α Update Loss Function

$$J(\alpha) = \alpha \cdot \mathbb{E}_{a_t \sim \pi_\phi} [-\log \pi_\phi(a_t|s_t) - \bar{\mathcal{H}}] \quad (15)$$

7.4 Dynamics of α Adaptation

To see how α changes, we take the derivative of $J(\alpha)$ with respect to α :

$$\frac{\partial J}{\partial \alpha} = \mathbb{E}_{a \sim \pi} [-\log \pi_\phi(a|s)] - \bar{\mathcal{H}} = \mathcal{H}(\pi) - \bar{\mathcal{H}}$$

In network training, we update parameters using **gradient descent**:

$$\alpha \leftarrow \alpha - \eta \frac{\partial J}{\partial \alpha}$$

Scenario 1: Actual Entropy is Too High

- Condition: $-\log \pi_\phi(a_t|s_t) > \bar{\mathcal{H}}$ on average, meaning $\mathcal{H}(\pi) > \bar{\mathcal{H}}$
- Gradient Sign: $\frac{\partial J}{\partial \alpha} > 0$ (positive)
- The Update: α **decreases**
- The Result: A lower α reduces the entropy penalty weight, allowing the agent to focus on exploitation.

Scenario 2: Actual Entropy is Too Low

- Condition: $-\log \pi_\phi(a_t|s_t) < \bar{\mathcal{H}}$ on average, meaning $\mathcal{H}(\pi) < \bar{\mathcal{H}}$
- Gradient Sign: $\frac{\partial J}{\partial \alpha} < 0$ (negative)
- The Update: α **increases**
- The Result: A higher α heavily penalizes predictable actions, forcing the policy to explore more.

Key Conclusion 7.1— Summary:

- $-\log \pi(a|s) > \bar{\mathcal{H}} \implies \text{Entropy is too high} \implies \alpha \text{ decreases.}$
- $-\log \pi(a|s) < \bar{\mathcal{H}} \implies \text{Entropy is too low} \implies \alpha \text{ increases.}$

7.5 Choosing Target Entropy $\bar{\mathcal{H}}$

$$\boxed{\bar{\mathcal{H}} = -\dim(\mathcal{A})} \tag{16}$$

where $\dim(\mathcal{A})$ is the number of dimensions in the action space.

8 The Partition Function $Z(s)$ in Practice

8.1 Physics Origin: Boltzmann Distribution

The concept of a **partition function** comes from statistical mechanics. In maximum entropy RL:

$$\pi^*(a|s) \propto \exp\left(\frac{1}{\alpha} Q(s, a)\right)$$

This is known as the **Boltzmann (or Gibbs) distribution**.

8.2 Why $Z(s)$ Cannot Be Computed Directly

Problem: Calculating the integral across continuous action spaces is **analytically intractable**:

$$Z(s) = \int_{\mathcal{A}} \exp\left(\frac{1}{\alpha} Q(s, a')\right) da'$$

8.3 The Gradient Vanishing Trick

Key Insight: We don't need to compute $Z(s)$ because it acts as a **constant** relative to the policy parameters ϕ .

Expanding the KL divergence objective:

$$J_{\pi}(\phi) = \mathbb{E}_{a \sim \pi_{\phi}} \left[\log \pi_{\phi}(a|s) - \frac{1}{\alpha} Q(s, a) + \log Z(s) \right]$$

When we take the gradient:

$$\boxed{\nabla_{\phi} \log Z(s) = 0} \tag{17}$$

The partition function naturally vanishes during differentiation.

Why the Information Projection is “Convenient”

With I-projection: $Z(s)$ becomes an additive constant $\implies$ vanishes during gradient update.

With alternative projections (e.g., reverse KL): $Z(s)$ would remain trapped inside an expectation, requiring the impossible integral.

9 Actor-Critic Architecture Analysis

9.1 Why Both Q-Network and V-Network? (Original Design)

In the original 2018 SAC paper, the authors included a **separate V-network** to reduce computational variance.

Benefit: By having a dedicated V-network, calculating the target for the Q-network doesn't require sampling an action a_{t+1} from the policy at every training step.

9.2 Why One Network Is Enough (SAC-v2)

In a subsequent revision (often called **SAC-v2**), the authors realized that maintaining three separate networks was **mathematically redundant**.

In modern SAC, we only use a Q-network. The target is calculated by sampling the next action directly:

$$J_Q(\theta) = \mathbb{E} \left[\frac{1}{2} (Q_{\theta}(s, a) - (r + \gamma(Q_{\bar{\theta}}(s', a') - \alpha \log \pi_{\phi}(a'|s'))))^2 \right]$$

9.3 The Circular Dependency Problem

Question: The update of V-network relies on Q, and Q relies on V. Isn't this severely unstable?

Answer: This circular dependency is called **bootstrapping**. Left unchecked, this causes values to explode or diverge.

SAC prevents this using a **Target Network** that is **frozen** or changing very slowly.

9.4 Target Network and Polyak Averaging

Polyak Averaging (Soft Updates):

$$\bar{\psi} \leftarrow \tau \psi + (1 - \tau) \bar{\psi} \quad (18)$$

where τ is a very small number (typically $\tau = 0.005$).

Table 4: Stabilization Effect

Network	Update Speed	Role
Q_θ (online)	Fast	Provides stable targets to the Q-network
$V_{\bar{\psi}}$ (target)	Slow (Polyak)	Tracks the true Q-value, preventing explosion

10 Twin Q-Networks and Overestimation Bias

10.1 The Maximization Bias Problem

A neural network $Q_\theta(s, a)$ naturally has estimation error:

$$Q_\theta(s, a) = Q^*(s, a) + \epsilon$$

where $\mathbb{E}[\epsilon] = 0$ (unbiased).

The Problem: In SAC, the actor is trained to maximize Q_θ . When selecting the action to maximize:

$$\mathbb{E} \left[\max_a Q_\theta(s, a) \right] \geq \max_a \mathbb{E}[Q_\theta(s, a)] = \max_a Q^*(s, a)$$

Due to **Jensen's Inequality**, the maximum of noisy estimates is **biased upward**.

10.2 Mathematical Analysis of Error Propagation

Step $t + 1$: The actor selects an action that overestimates the value of state s_{t+1} .

Step t : The Q-network updates its value for s_t by bootstrapping off that **inflated** s_{t+1} target.

The Loop: Overestimation error cascades backward and multiplies, causing Q-values to grow infinitely large.

10.3 The Minimum Q-Value Trick (Clipped Double-Q)

To break this loop, SAC adopts the **Clipped Double-Q Learning** mechanism. We initialize two **independent** Q-networks:

$$Q_{\theta_1}(s, a) \quad \text{and} \quad Q_{\theta_2}(s, a)$$

When calculating the target, we take the **minimum**:

$$y = r + \gamma \left(\min_i Q_{\tilde{\theta}_i}(s', a') - \alpha \log \pi_\phi(a'|s') \right) \quad (19)$$

Why the Minimum Works: Since Q_{θ_1} and Q_{θ_2} are independent, their estimation errors are also independent. By taking the minimum, the algorithm **filters out positive noise peaks**.

Key Conclusion 10.1— Why underestimation is safer than overestimation:

An underestimation bias is vastly safer for a reinforcement learning agent. If the agent underestimates a state slightly, it simply continues to explore. But if it overestimates, it will get stuck forever repeating a bad action, believing it is optimal.

11 Off-Policy vs On-Policy Analysis

11.1 Why SAC Can Be Off-Policy

The Critic Doesn't Care Who Took the Action: The Soft Bellman Equation states a fundamental truth: if you are in state s_t and execute action a_t , the resulting reward and next state s_{t+1} depend **entirely on the physics of the environment**. It doesn't matter if that action was chosen by the current smart policy or a random policy from training hour one.

The Actor Evaluates Old States with New Actions:

- States s are sampled from the **historical replay buffer** $\mathcal{D}$.
- Actions a are sampled from the **current policy** π_ϕ .

11.2 Why PPO Is Inherently On-Policy

PPO belongs to the **Policy Gradient** family. To optimize using data collected by an older policy π_{old} , policy gradient methods use **Importance Sampling**:

$$r_t(\theta) = \frac{\pi_{\text{old}}(a_t|s_t)}{\pi_\theta(a_t|s_t)}$$

Problem: For this correction to have low variance, π_θ must be **incredibly close** to π_{old} . If you feed PPO data from a replay buffer collected 100 iterations ago, the ratio $r_t(\theta)$ explodes toward infinity.

11.3 Key Architectural Differences

Table 5: SAC vs PPO Comparison

Property	SAC	PPO
Policy Type	Off-policy	On-policy
Data Reuse	Replay buffer (any past data)	Must discard after update
Value Estimation	Bellman equation with critics	GAE
Stability	Target networks + Twin-Q	Clipping mechanism
Sample Efficiency	High (reuses data)	Low (wastes data)

12 Implementation Details and Engineering Tricks

12.1 Why Predict `log_std` Instead of `std`

A standard deviation σ must be **strictly positive** ($\sigma > 0$). If your neural network outputs `std` directly, it could output a negative number during early training, crashing the code.

Solution: Force the network to predict $\log \sigma$. When we compute $\sigma = \exp(\log \sigma)$, the exponential function **mathematically guarantees** a strictly positive value.

12.2 Clamping `log_std`: `[-20, 2]` Rationale

Exploding Variance (`log_std_max = 2`): If $\log \sigma = 20$, then $\sigma = \exp(20) \approx 4.8 \times 10^8$. Actions become pure random noise.

Numerical Underflow (`log_std_min = -20`): If $\log \sigma = -50$, then $\sigma = \exp(-50) \approx 2.06 \times 10^{-9}$. This causes NaN errors when calculating $\log \pi(a|s)$.

Clamping $\log \sigma$ between $[-20, 2]$ keeps σ safely bounded.

12.3 The `.sum(dim=-1, keepdim=True)` Operation

When an environment has multiple action dimensions, the actor network outputs a mean μ_i and standard deviation σ_i for **each** individual dimension.

SAC assumes a **Diagonal Gaussian policy**: Action dimensions are statistically independent.

Joint Probability: The joint probability of multiple independent events is the **product** of their individual probabilities. In log-space, products become sums:

$$\log P(a_1, a_2, \dots, a_D) = \sum_{i=1}^D \log P(a_i)$$

12.4 Exploration vs Evaluation Mode

select_action function

```
def select_action(self, state, evaluate=False):
    state = torch.FloatTensor(state).unsqueeze(0)
    if evaluate:
        with torch.no_grad():
            mu, _ = self.actor(state)
            action = torch.tanh(mu)
            return action.numpy()[0]
    else:
        with torch.no_grad():
            action, _ = self.actor.sample(state)
            return action.numpy()[0]
```

When evaluate=False (Training Mode): The code calls `self.actor.sample(state)`. This uses the reparameterization trick to sample an action randomly. **Purpose:** Adds exploratory noise.

When evaluate=True (Testing Mode): The code bypasses σ and sampling entirely, extracting only the mean μ . **Purpose:** Makes the policy completely **deterministic** for testing.

13 Complete PyTorch Implementation

13.1 Replay Buffer

```
1 class ReplayBuffer:
2     def __init__(self, capacity, state_dim, action_dim):
3         self.capacity = capacity
4         self.ptr = 0
5         self.size = 0
6
7         self.states = np.zeros((capacity, state_dim), dtype=np.
8                                 float32)
9         self.actions = np.zeros((capacity, action_dim), dtype=np.
10                                float32)
11        self.rewards = np.zeros((capacity, 1), dtype=np.float32)
12        self.next_states = np.zeros((capacity, state_dim), dtype=
13                                     np.float32)
14        self.dones = np.zeros((capacity, 1), dtype=np.float32)
15
16    def push(self, state, action, reward, next_state, done):
17        self.states[self.ptr] = state
18        self.actions[self.ptr] = action
19        self.rewards[self.ptr] = reward
20        self.next_states[self.ptr] = next_state
21        self.dones[self.ptr] = done
22
23        self.ptr = (self.ptr + 1) % self.capacity
```

```

21         self.size = min(self.size + 1, self.capacity)
22
23     def sample(self, batch_size):
24         ind = np.random.randint(0, self.size, size=batch_size)
25         return (
26             torch.FloatTensor(self.states[ind]),
27             torch.FloatTensor(self.actions[ind]),
28             torch.FloatTensor(self.rewards[ind]),
29             torch.FloatTensor(self.next_states[ind]),
30             torch.FloatTensor(self.dones[ind])
31         )

```

Listing 1: Replay Buffer

13.2 Network Definitions

```

1  class CriticNetwork(nn.Module):
2      """Twin Q-networks inside a single module for clean
3      optimization."""
4      def __init__(self, state_dim, action_dim, hidden_dim=256):
5          super(CriticNetwork, self).__init__()
6          # Q1
7          self.q1 = nn.Sequential(
8              nn.Linear(state_dim + action_dim, hidden_dim), nn.
9              ReLU(),
10             nn.Linear(hidden_dim, hidden_dim), nn.ReLU(),
11             nn.Linear(hidden_dim, 1)
12         )
13         # Q2
14         self.q2 = nn.Sequential(
15             nn.Linear(state_dim + action_dim, hidden_dim), nn.
16             ReLU(),
17             nn.Linear(hidden_dim, hidden_dim), nn.ReLU(),
18             nn.Linear(hidden_dim, 1)
19         )
20
21     def forward(self, state, action):
22         sa = torch.cat([state, action], dim=-1)
23         return self.q1(sa), self.q2(sa)
24
25 class ActorNetwork(nn.Module):
26     def __init__(self, state_dim, action_dim, hidden_dim=256,
27                 log_std_min=-20, log_std_max=2):
28         super(ActorNetwork, self).__init__()
29         self.log_std_min = log_std_min
30         self.log_std_max = log_std_max
31
32         self.net = nn.Sequential(
33             nn.Linear(state_dim, hidden_dim), nn.ReLU(),

```

```

31         nn.Linear(hidden_dim, hidden_dim), nn.ReLU()
32     )
33     self.mu = nn.Linear(hidden_dim, action_dim)
34     self.log_std = nn.Linear(hidden_dim, action_dim)
35
36     def forward(self, state):
37         x = self.net(state)
38         mu = self.mu(x)
39         log_std = self.log_std(x)
40         log_std = torch.clamp(log_std, self.log_std_min, self.
            log_std_max)
41         return mu, log_std
42
43     def sample(self, state):
44         mu, log_std = self.forward(state)
45         std = log_std.exp()
46
47         # Reparameterization trick
48         normal = Normal(mu, std)
49         x_t = normal.rsample()
50         action = torch.tanh(x_t)
51
52         # Tanh Log-Likelihood Correction
53         log_prob = normal.log_prob(x_t) - torch.log(1 - action.
            pow(2) + 1e-6)
54         log_prob = log_prob.sum(dim=-1, keepdim=True)
55
56         return action, log_prob

```

Listing 2: Network Definitions

13.3 SAC Agent

```

1 class SACTAgent:
2     def __init__(self, state_dim, action_dim, lr=3e-4, gamma
        =0.99, tau=0.005):
3         self.gamma = gamma
4         self.tau = tau
5         self.action_dim = action_dim
6
7         # Critics
8         self.critic = CriticNetwork(state_dim, action_dim)
9         self.critic_target = CriticNetwork(state_dim, action_dim)
10        self.critic_target.load_state_dict(self.critic.state_dict
            ())
11        self.critic_opt = optim.Adam(self.critic.parameters(), lr
            =lr)
12
13        # Actor
14        self.actor = ActorNetwork(state_dim, action_dim)

```

```

15     self.actor_opt = optim.Adam(self.actor.parameters(), lr=
        lr)
16
17     # Automatic Entropy Tuning
18     self.target_entropy = -action_dim
19     self.log_alpha = torch.zeros(1, requires_grad=True)
20     self.alpha_opt = optim.Adam([self.log_alpha], lr=lr)
21
22     @property
23     def alpha(self):
24         return self.log_alpha.exp()
25
26     def select_action(self, state, evaluate=False):
27         state = torch.FloatTensor(state).unsqueeze(0)
28         if evaluate:
29             with torch.no_grad():
30                 mu, _ = self.actor(state)
31                 action = torch.tanh(mu)
32                 return action.numpy()[0]
33         else:
34             with torch.no_grad():
35                 action, _ = self.actor.sample(state)
36                 return action.numpy()[0]
37
38     def update(self, memory, batch_size):
39         state, action, reward, next_state, done = memory.sample(
            batch_size)
40
41         # 1. Update Critic
42         with torch.no_grad():
43             next_action, next_log_prob = self.actor.sample(
                next_state)
44             target_q1, target_q2 = self.critic_target(next_state,
                next_action)
45             target_q = torch.min(target_q1, target_q2) - self.
                alpha * next_log_prob
46             expected_q = reward + (1 - done) * self.gamma *
                target_q
47
48             curr_q1, curr_q2 = self.critic(state, action)
49             critic_loss = nn.MSELoss()(curr_q1, expected_q) + nn.
                MSELoss()(curr_q2, expected_q)
50
51             self.critic_opt.zero_grad()
52             critic_loss.backward()
53             self.critic_opt.step()
54
55         # 2. Update Actor
56         new_action, log_prob = self.actor.sample(state)
57         q1, q2 = self.critic(state, new_action)
58         min_q = torch.min(q1, q2)

```

```

59
60     actor_loss = (self.alpha * log_prob - min_q).mean()
61
62     self.actor_opt.zero_grad()
63     actor_loss.backward()
64     self.actor_opt.step()
65
66     # 3. Update Alpha
67     alpha_loss = -(self.log_alpha * (log_prob + self.
        target_entropy).detach()).mean()
68
69     self.alpha_opt.zero_grad()
70     alpha_loss.backward()
71     self.alpha_opt.step()
72
73     # 4. Soft Update Targets
74     for param, target_param in zip(self.critic.parameters(),
        self.critic_target.parameters()):
75         target_param.data.copy_(self.tau * param.data + (1 -
            self.tau) * target_param.data)

```

Listing 3: SAC Agent

14 Practical Projects

14.1 Quadcopter / Drone Control

Environment: gym-pybullet-drones or AirSim

Why SAC fits: Drones require precise, continuous motor RPM adjustments. SAC’s entropy exploration naturally discovers smoother control trajectories.

14.2 Autonomous Driving & Lane Keeping

Environment: CARLA Simulator or highway-env

Why SAC fits: Driving relies on smooth throttle, braking, and steering inputs. The maximum entropy framework prevents the vehicle from getting trapped in local minima.

14.3 Bipedal Locomotion

Environment: MuJoCo (Ant-v4, Walker2d-v4, or Humanoid-v4)

Why SAC fits: SAC is one of the few algorithms robust enough to successfully map coordinated continuous joint movements for top-heavy bipeds.

14.4 Financial Portfolio Optimization

Environment: FinRL framework or custom Gymnasium asset tracking code.

Why SAC fits: The action space is a softmax probability vector over continuous asset weights. SAC’s entropy exploration acts as an **implicit risk-management tool**.

15 Training Diagnostics

15.1 Understanding Reward Vibration

The “huge vibration” in rewards during training is actually **not a failure to converge**. Your SAC agent has likely already learned the optimal policy!

The vibration is a combined result of how the environment is designed and how SAC samples actions during training.

15.2 The Role of Randomized Initial States

Your code is running on Pendulum-v1. In this environment, every single time an episode resets, the pendulum is spawned at a **completely random angle** ($\theta \in [-\pi, \pi]$) with random velocity.

- **Single-Digit Episodes (e.g., -1.59):** The pendulum happened to spawn almost perfectly upright.
- **High-Negative Episodes (e.g., -115.40):** The pendulum spawned hanging straight down.

15.3 How to Verify True Convergence

To see if your agent has actually converged, test it **deterministically**:

```

1 eval_reward = 0
2 eval_state, _ = env.reset()
3 eval_done = False
4
5 while not eval_done:
6     action = agent.select_action(eval_state, evaluate=True)
7     scaled_action = action * env.action_space.high[0]
8
9     next_state, reward, terminated, truncated, _ = env.step(
10         scaled_action)
11     eval_done = terminated or truncated
12     eval_state = next_state
13     eval_reward += reward
14 print(f"=== DETERMINISTIC EVALUATION REWARD: {eval_reward:.2f}
    ===")

```

The deterministic evaluation rewards will be **consistently high and smooth**.

16 Study Quiz

16.1 Question 1: Core Optimization Objective

What is the core optimization objective of SAC that distinguishes it from standard model-free RL?

- A) Minimizing the tracking error of a reference trajectory.
- B) Maximizing expected return while also maximizing policy entropy.
- C) Maximizing the deterministic reward of the environment.
- D) Minimizing the variance of the state-value function updates.

Answer: B

SAC augments the standard reward maximization objective with an entropy term. This framework encourages exploration and robustness by rewarding the policy for acting randomly.

16.2 Question 2: The Temperature Parameter (α)

What happens if α is set to 0 in SAC?

- A) The policy becomes purely random, prioritizing maximum entropy.
- B) The critic updates stop because the soft Q-value function becomes undefined.
- C) The algorithm reduces to a standard, non-entropy-augmented RL algorithm.
- D) The actor will completely stop updating because gradients vanish.

Answer: C

Setting $\alpha = 0$ removes the entropy regularization term completely. The objective function simplifies back to standard expected reward maximization.

16.3 Question 3: Twin Q-Functions

Why does SAC use two Q-networks?

- A) To mitigate overestimation bias by taking the minimum of the two targets.
- B) To separately model the rewards from the environment and the intrinsic entropy rewards.
- C) To allow simultaneous on-policy and off-policy evaluation.
- D) To compute the policy gradient using a contrastive loss between two independent critics.

Answer: A

Taking the minimum of two independent Q-value targets bounds the overestimation bias that commonly plagues actor-critic methods.

16.4 Question 4: Continuous Action Spaces

How does the actor network output actions for stochastic exploration?

- A) It outputs a deterministic action vector combined with Ornstein-Uhlenbeck noise.
- B) It outputs discrete logits representing a softmax probability distribution.
- C) It outputs a single scalar value corresponding to the expected value.
- D) It outputs the mean and standard deviation of a Gaussian distribution.

Answer: D

The policy is parameterized as a squashed Gaussian distribution. The neural network outputs the mean and variance for each action dimension.

16.5 Question 5: Gradient Variance

Which technique is used in SAC to compute low-variance gradients for the policy?

- A) The REINFORCE score function estimator.
- B) The reparameterization trick (pathwise derivative estimator).
- C) Generalized Advantage Estimation (GAE).
- D) Importance sampling correction from an on-policy buffer.

Answer: B

The reparameterization trick expresses the stochastic action as a deterministic function of the policy parameters and an independent noise variable.

16.6 Question 6: Soft Q-Value Target

What is the target value function y for a transition (s, a, r, s') in SAC?

- A) $y = r + \gamma[\min_i Q_{\theta_i}^{\text{target}}(s', a') - \alpha \log \pi_\phi(a'|s')]$
- B) $y = r + \gamma \max_{a'} [\min_i Q_{\theta_i}(s', a')]$
- C) $y = r + \alpha \log \pi_\phi(a|s) + \gamma \max_i Q_{\theta_i}(s', a')$
- D) $y = r + \gamma[\frac{1}{2}Q_{\theta_1}(s', a') + Q_{\theta_2}(s', a')]$

Answer: A

The target value incorporates the reward, discounted minimum soft Q-value, and subtracts the scaled log-probability for the entropy bonus.

16.7 Question 7: Automatic Temperature Adjustment

How is α handled in modern implementations of SAC?

- A) Decayed according to a predefined exponential schedule.
- B) Fixed as a hyperparameter that must be manually tuned.
- C) Automatically adjusted over time by optimizing a loss constrained to a target entropy.
- D) Learned directly as an extra output head of the Critic network.

Answer: C

Modern SAC automatically tunes α dynamically by formulating a constrained optimization problem.

16.8 Question 8: Algorithm Type

Is SAC on-policy or off-policy, and what component enables this?

- A) On-policy, enabled by calculating the soft policy gradient directly from current rollouts.

- B) Off-policy, enabled by a replay buffer that stores past experiences.
- C) Off-policy, enabled by updating the policy using importance sampling weights.
- D) On-policy, enabled by maintaining synchronized target networks.

Answer: B

SAC is an off-policy algorithm. It utilizes a replay buffer to store and reuse historical data transitions.

16.9 Question 9: Benefits of Maximum Entropy

What is a primary benefit of maximizing entropy in SAC?

- A) It encourages the policy to explore more widely and prevent premature convergence.
- B) It ensures that the critic network converges faster.
- C) It completely eliminates the need for target networks.
- D) It reduces the memory requirements of the replay buffer.

Answer: A

Explicitly maximizing entropy ensures the agent keeps its options open and continues exploring alternative actions.

16.10 Question 10: Target Network Updates

How are the parameters of the target Q-networks updated in SAC?

- A) By copying the weights every fixed number of steps (hard updates).
- B) Directly via gradient descent using the mean squared error.
- C) They are not updated online; trained offline after collecting data.
- D) Through Polyak averaging (soft updates) using a small coefficient τ .

Answer: D

Target networks in SAC track the main online Q-networks via soft updates: $\theta_{\text{target}} \leftarrow \tau\theta + (1 - \tau)\theta_{\text{target}}$.

A Appendix A: Mathematical Foundations

A.1 Kullback-Leibler Divergence Properties

The KL divergence $D_{\text{KL}}(P||Q)$ measures the “distance” from distribution Q to P :

$$D_{\text{KL}}(P||Q) = \mathbb{E}_{x \sim P} \left[\log \frac{P(x)}{Q(x)} \right]$$

Key Properties:

1. **Non-negativity:** $D_{\text{KL}}(P||Q) \geq 0$
2. **Zero iff identical:** $D_{\text{KL}}(P||Q) = 0 \iff P = Q$
3. **Asymmetry:** $D_{\text{KL}}(P||Q) \neq D_{\text{KL}}(Q||P)$ in general

A.2 Change-of-Variables Formula (General Form)

For a transformation $a = f(u)$ where f is invertible:

$$\pi_A(a) = \pi_U(f^{-1}(a)) \cdot \left| \det \left(\frac{df^{-1}}{da} \right) \right|$$

A.3 Softplus Function Stable Implementation

The softplus function:

$$\text{softplus}(x) = \log(1 + e^x)$$

Connection to tanh correction:

$$\log(1 - \tanh^2(u)) = 2 \log 2 - u - \text{softplus}(-2u)$$

B Appendix B: Symbol Table

Table 6: Symbol Table

Symbol	Meaning
π	Policy function
$\pi_\phi(a s)$	Policy parameterized by neural network with parameters ϕ
ρ_π	State-action trajectory distribution under policy π
α	Temperature parameter controlling entropy-reward trade-off
$\mathcal{H}(\pi(\cdot s))$	Shannon entropy of policy at state s
$\bar{\mathcal{H}}$	Target entropy threshold
$Q(s, a)$	Soft Q-function: value of taking action a in state s
$V(s)$	Soft V-function: value of being in state s
γ	Discount factor
$\mathcal{T}^\pi$	Soft Bellman backup operator
$Z(s)$	Partition function for normalizing the Boltzmann distribution
$Q_\theta(s, a)$	Q-function parameterized by neural network with parameters θ
$Q_{\bar{\theta}}$	Target Q-network (slowly updated)
ϵ	Random noise vector for reparameterization trick
$\mu_\phi(s)$	Mean of the Gaussian policy

Symbol	Meaning
$\sigma_\phi(s)$	Standard deviation of the Gaussian policy
τ	Polyak averaging coefficient for soft target updates
$\mathcal{D}$	Replay buffer containing past transitions
$D_{\text{KL}}(P\ Q)$	Kullback-Leibler divergence from Q to P
$\text{softplus}(x)$	$\log(1 + e^x)$

C Appendix C: Core Formula Summary

Maximum Entropy Objective:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^T r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot|s_t)) \right]$$

Soft Bellman Equation:

$$Q(s_t, a_t) = r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1} \sim p} [V(s_{t+1})]$$

Soft V-Function:

$$V(s_t) = \mathbb{E}_{a_t \sim \pi} [Q(s_t, a_t) - \alpha \log \pi(a_t|s_t)]$$

Reparameterization Trick:

$$a = f_\phi(\epsilon; s) = \tanh(\mu_\phi(s) + \sigma_\phi(s) \odot \epsilon), \quad \epsilon \sim \mathcal{N}(0, I)$$

Tanh Log-Likelihood Correction:

$$\log \pi(a|s) = \log \mu(u|s) - \sum_{i=1}^D \log(1 - \tanh^2(u_i))$$

Stable Tanh Correction:

$$\log(1 - \tanh^2(u)) = 2 \log 2 - u - \text{softplus}(-2u)$$

Actor Loss:

$$J_\pi(\phi) = \mathbb{E}_{s \sim \mathcal{D}, a \sim \pi_\phi} [\alpha \log \pi_\phi(a|s) - Q_\theta(s, a)]$$

Critic Loss:

$$J_Q(\theta) = \mathbb{E}_{(s, a, r, s') \sim \mathcal{D}} \left[\frac{1}{2} (Q_\theta(s, a) - (r + \gamma(Q_{\bar{\theta}}(s', a') - \alpha \log \pi_\phi(a'|s'))))^2 \right]$$

Alpha Loss (Automatic Tuning):

$$J(\alpha) = \alpha \cdot \mathbb{E}_{a_t \sim \pi_\phi} [-\log \pi_\phi(a_t | s_t) - \bar{\mathcal{H}}]$$

Polyak Averaging (Soft Updates):

$$\bar{\theta} \leftarrow \tau \theta + (1 - \tau) \bar{\theta}$$

Twin Q-Minimum Trick:

$$y = r + \gamma \left(\min_i Q_{\bar{\theta}_i}(s', a') - \alpha \log \pi_\phi(a' | s') \right)$$

Author Information

Name: 千喜 Qx

Date: 2026/06/09

Based on materials from: Haarnoja et al. (2018) and subsequent SAC-v2 revisions

Introduction: 一位刚刚入门具身科研领域的学徒，未来想长期从事具身方向的科研和创业，用机器人替代人类重复的体力劳动。欢迎大家加我 vx 交朋友，我很丰富，无法简介;)

vx: Y1832800
